

- [Advanced Threat Detection System - Master Guide](#)
 - [Table of Contents](#)
- [1. Introduction & Overview](#)
 - [System Overview](#)
 - [Key Features](#)
 - [Security Monitoring](#)
 - [Integrated Platform](#)
 - [ELK Stack Powered](#)
 - [Quick Start Summary](#)
 - [Tech Stack](#)
 - [Beginner's Guide](#)
 - [Welcome! This Guide is for Complete Beginners](#)
 - [What is this System? \(The Big Picture\)](#)
 - [What are the Main Parts? \(Simple Explanations\)](#)
 - [1. FastAPI \(The Front Door\)](#)
 - [2. PostgreSQL \(The Filing Cabinet\)](#)
 - [3. Redis \(The Sticky Notes\)](#)
 - [4. Elasticsearch \(The Smart Librarian\)](#)
 - [5. Logstash \(The Translator\)](#)
 - [6. Kibana \(The Artist\)](#)
 - [7. Beats \(The Security Guards\)](#)
 - [How Do They Work Together? \(The Complete Story\)](#)
 - [Step 1: The Attack Begins](#)
 - [Step 2: Information Gets Processed](#)
 - [Step 3: The Smart Analysis](#)
 - [Step 4: The Alert](#)
 - [Step 5: The Response](#)
 - [Real-World Examples \(What This Looks Like\)](#)
 - [Example 1: A Normal Day](#)
 - [Example 2: A Cyber Attack](#)
 - [What Can You Do With This System?](#)
 - [As a Regular User:](#)
 - [As a Security Administrator:](#)
 - [As a System Administrator:](#)
 - [Understanding the Security Features](#)
 - [What Attacks Can This System Detect?](#)
 - [Risk Scoring System \(How Dangerous is Each Event?\)](#)

- Common Terms Explained
 - Technical Terms Made Simple
 - Security Terms Made Simple
- 2. System Architecture
 - High-Level Architecture
 - What Does Each Part Do?
 - 1. The Web App (FastAPI)
 - 2. Threat Detection Engine
 - 3. ELK Stack (The Brain)
 - 4. Database Cluster
 - 5. Beats (Data Collectors)
 - How Data Flows Through the System
 - Detailed Data Flow
 - Physical Architecture (How Components Connect)
 - Component Interactions
 - Complete Data Flow Diagrams
 - Data Collection Flow
 - From Computer Events to Security Alerts
 - Detailed Component Interactions
 - 1. User Authentication Flow
 - 2. Todo CRUD Operations Flow
 - 3. Security Threat Detection Flow
 - ELK Stack Guide
 - What is the ELK Stack?
 - Elasticsearch - The Data Storage Expert
 - What is Elasticsearch?
 - How Does it Work?
 - Key Features
 - Configuration in Our System
 - Data Structure (Indices)
 - Example Data in Elasticsearch
 - Logstash - The Data Processor
 - What is Logstash?
 - How Does it Work?
 - The Pipeline Process
 - Configuration Ports
 - Processing Pipeline Example
 - Kibana - The Visualization Dashboard

- What is Kibana?
 - How Does it Work?
 - Key Features
 - Dashboard Examples
 - Security Operations Dashboard
 - Geographic Threat Map
 - Configuration
 - Creating Security Index Patterns
 - How ELK Components Work Together
 - Data Flow Example: Detecting a Brute Force Attack
- 4. Setup & Configuration
 - Complete Setup Guide
 - Prerequisites
 - System Requirements
 - Required Software
 - Installation Steps
 - Step 1: Clone the Repository
 - Step 2: Configure Environment Variables
 - Step 3: Start the System
 - Step 4: Initialize the Database
 - Step 5: Verify Installation
 - Security Credentials & Access
 - Default Credentials (Development Only)
 - Access Points
 - Security Features Active
 - Production Security Checklist
 - Troubleshooting Common Issues
 - Issue 1: Services Won't Start
 - Issue 2: Elasticsearch Won't Start
 - Issue 3: Kibana Shows No Data
 - Issue 4: Database Connection Failed
 - Emergency Reset
- 5. Deployment Guides
 - Kubernetes Deployment
 - Quick Start (Automated)
 - Manual Deployment Steps
 - 1. Cluster Setup
 - 2. Build & Load Images

- 3. Deploy Resources
 - Accessing Applications
 - Architecture
- AWS EC2 Deployment
 - Quick Start (One-Command)
 - Manual Setup Steps
 - Access Points (AWS)
 - Maintenance
- 6. Operations & API
 - Monitoring & APT Detection
 - Log Sources & Detection Capabilities
 - Risk Scoring System
 - APT Detection Scenarios
 - 1. Credential Stuffing
 - 2. Lateral Movement
 - 3. Data Exfiltration
 - Alerting Rules (ES|QL)
 - 1. Brute Force Detection
 - 2. PowerShell Attack Detection
 - 3. Data Exfiltration
 -  API Documentation
 - Authentication
 - Key Endpoints
 - User Management
 - Security Monitoring
 - System Health
 - Example Usage

Advanced Threat Detection System - Master Guide

Welcome to the complete documentation for the Advanced Threat Detection System. This master guide consolidates all project documentation into a single, sequential manual designed to take you from zero knowledge to full mastery of the system.

Table of Contents

1. Introduction & Overview

- System Overview
- Key Features
- Beginner's Guide

2. System Architecture

- High-Level Architecture
- Component Interactions
- Database Architecture

3. Core Components Deep Dive

- ELK Stack Guide
- Elasticsearch
- Logstash
- Kibana
- Beats (Data Collectors)

4. Installation & Setup

- Complete Setup Guide
- Security Credentials
- Troubleshooting

5. Deployment Guides

- Kubernetes Deployment
- AWS Deployment

6. Operations & API

- Monitoring & APT Detection
- Alerting Rules (ES|QL)
- API Documentation

1. Introduction & Overview

System Overview

Enterprise-grade security monitoring solution with real-time APT detection using ELK Stack

The Advanced Threat Detection System is a comprehensive cybersecurity platform that combines task management with advanced threat detection capabilities, powered by the ELK Stack (Elasticsearch, Logstash, Kibana) for real-time security monitoring and APT (Advanced Persistent Threat) detection.

Key Features

Security Monitoring

- **Real-time Threat Detection** - Automated detection of brute force attacks, data exfiltration, and APT activities
- **PowerShell Attack Monitoring** - Detection of suspicious command execution and encoded scripts
- **Risk-based Alerting** - Intelligent threat scoring and prioritized notifications
- **Cross-system Correlation** - APT kill-chain analysis across multiple data sources

Integrated Platform

- **Todo Management** - Secure task management with priority levels and due dates
- **User Authentication** - JWT-based secure authentication system
- **Real-time Dashboard** - Live security monitoring and analytics
- **API Integration** - RESTful API for seamless frontend-backend communication

ELK Stack Powered

- **Elasticsearch** - High-performance search and analytics for security data
- **Logstash** - Real-time log processing and threat enrichment
- **Kibana** - Interactive security dashboards and visualizations

Quick Start Summary

1. Clone the repository

```
git clone https://github.com/rohansen856/elk-stack-monitoring
cd elk-stack-monitoring
```

2. Configure environment

```
cp .env.example .env
```

3. Start all services

```
docker-compose up -d
```

4. Access the applications

- **Frontend Dashboard:** <http://localhost:3000>
- **API Documentation:** <http://localhost:8000/docs>
- **Kibana Security Dashboard:** <http://localhost:5601>

Tech Stack

Component	Technology	Purpose
Frontend	Next.js 16, TypeScript, Tailwind CSS	User interface and dashboards
Backend	FastAPI, Python 3.11+, Pydantic	API server and business logic
Database	PostgreSQL, Redis, SQLAlchemy	Data persistence and caching
Security	Elasticsearch, Logstash, Kibana	Security monitoring and analytics
Monitoring	Filebeat, Metricbeat, Prometheus	System and security metrics
Deployment	Docker, Docker Compose	Containerized deployment

Beginner's Guide

Welcome! This Guide is for Complete Beginners

If you're new to security systems, databases, or even computers in general, this guide will help you understand our Advanced Threat Detection System. We'll explain

everything using simple analogies and examples that anyone can understand.

What is this System? (The Big Picture)

Imagine you have a **smart house security system**. Our Advanced Threat Detection System works similarly, but instead of protecting your house, it protects computer networks from bad guys (hackers).

Your House Security System:

- Security cameras watch every room
- Alarm goes off when someone breaks in
- You get notifications on your phone
- Security company responds to threats

Our Threat Detection System:

- Log collectors watch every computer activity
- Alerts trigger when hackers attack
- Slack/email notifications sent to security team
- Automated analysis responds to threats

What are the Main Parts? (Simple Explanations)

1. FastAPI (The Front Door)

What it's like: The receptionist at a doctor's office **What it does:**

- Greets people (users) when they arrive
- Checks their appointment (login credentials)
- Directs them to the right place (API endpoints)
- Keeps track of who's visiting (user sessions)

Example: When you visit a website and log in, FastAPI is what handles your login and shows you the correct page.

2. PostgreSQL (The Filing Cabinet)

What it's like: A massive, organized filing cabinet in an office **What it does:**

- Stores all important information permanently

- Keeps user accounts and their todo lists
- Never loses information, even if power goes out
- Organized so you can find things quickly

Example: Like how your bank keeps records of all your transactions, PostgreSQL keeps records of all users and their data.

3. Redis (The Sticky Notes)

What it's like: Super-fast sticky notes on your desk **What it does:**

- Remembers things temporarily for quick access
- Stores information that's used often
- Much faster than checking the filing cabinet
- Information disappears after a while (which is okay!)

Example: Like how you write your friend's phone number on a sticky note so you don't have to look it up in your phone book every time.

4. Elasticsearch (The Smart Librarian)

What it's like: A librarian who has read every book and can instantly find any information **What it does:**

- Stores millions of security events (logs)
- Can search through everything in seconds
- Finds patterns and connections between events
- Answers questions like "Show me all attacks from China last week"

Example: Like Google, but for security events instead of web pages.

5. Logstash (The Translator)

What it's like: A translator who speaks every language **What it does:**

- Takes messy, different-formatted logs from various sources
- Cleans them up and translates them into a standard format
- Adds extra useful information (like location data)
- Sends clean, organized data to Elasticsearch

Example: Like having a translator who takes letters written in different languages and handwriting styles, and rewrites them all in clear, typed English.

6. Kibana (The Artist)

What it's like: An artist who draws pictures to explain complex data **What it does:**

- Creates beautiful charts and graphs
- Shows security information visually
- Makes complex data easy to understand
- Updates in real-time as new threats are detected

Example: Like the weather app that shows you temperature graphs and rain maps instead of just numbers.

7. Beats (The Security Guards)

What it's like: Security guards with walkie-talkies stationed around a building **What they do:**

- **Filebeat:** Reads log files and reports events
- **Metricbeat:** Monitors system performance (like a health monitor)
- **Winlogbeat:** Specifically watches Windows computers
- Send all information to Logstash for processing

Example: Like security guards who walk around a building, write down everything they see on clipboards, and radio the information to the security office.

How Do They Work Together? (The Complete Story)

Let's follow what happens when someone tries to hack into our system:

Step 1: The Attack Begins

```
Hacker: Tries to log in with wrong password 5 times
Computer: Writes this in the log file
Filebeat: "I see something suspicious happening!"
```

Step 2: Information Gets Processed

```
Filebeat: Sends the log to Logstash
Logstash: "Let me clean this up and add location data"
Logstash: "This IP address is from China - that's suspicious!"
Logstash: Sends processed data to Elasticsearch
```

Step 3: The Smart Analysis

```
Elasticsearch: Stores the security event
Threat Engine: "Let me check if this matches attack patterns"
Threat Engine: "Yes! This looks like a brute force attack!"
```

Step 4: The Alert

```
Alerting System: "DANGER! Sending alerts!"
Slack: " SECURITY ALERT: Brute force attack detected"
Email: Sends detailed report to security team
Kibana: Updates dashboards with new threat
```

Step 5: The Response

```
Security Team: Gets notifications
Security Team: Checks Kibana dashboard
Security Team: Blocks the attacker's IP address
Problem Solved!
```

Real-World Examples (What This Looks Like)

Example 1: A Normal Day

```
9:00 AM - John logs into the system
- Filebeat: Collects the login event
- Logstash: Processes it as "normal login"
- Elasticsearch: Stores it with low risk score
- No alerts sent (everything is normal)

10:30 AM - Jane creates a new todo
```

- FastAPI: Handles the request
- PostgreSQL: Saves the new todo
- Redis: Clears Jane's cache so she sees fresh data
- Activity logged for monitoring

Example 2: A Cyber Attack

```
2:17 AM - Someone from Russia tries logging in as "admin"
- Filebeat: Collects failed login attempt
- Logstash: Adds geographic data (Russia = suspicious at 2 AM)
- Elasticsearch: Stores with medium risk score

2:18 AM - Same IP tries 4 more times
- Threat Engine: "This pattern matches brute force attack!"
- Risk score jumps to 9/10
- Immediate alerts sent to security team

2:19 AM - Security team blocks the IP
- Attack stopped
- System protected
- Incident documented for future reference
```

What Can You Do With This System?

As a Regular User:

- Create and manage your todo lists
- Your account is protected by advanced security
- Your data is safely stored and backed up
- You get fast responses thanks to caching

As a Security Administrator:

- Monitor all security events in real-time
- See beautiful dashboards showing threat landscapes
- Get immediate alerts when attacks happen
- Track security trends and improvements over time
- See geographic maps of where attacks come from

As a System Administrator:

- Monitor system performance and health

- Scale individual components as needed
- See detailed metrics and logs
- Configure and tune the threat detection rules

Understanding the Security Features

What Attacks Can This System Detect?

1. Brute Force Attacks

- **What it is:** Someone trying to guess passwords by trying many combinations
- **How we detect it:** Look for many failed login attempts from the same IP
- **Example:** 10 failed logins in 5 minutes = ALERT!

2. Data Theft (Exfiltration)

- **What it is:** Hackers stealing large amounts of data
- **How we detect it:** Monitor unusual large data transfers
- **Example:** 500MB uploaded to external server at 3 AM = ALERT!

3. PowerShell Attacks

- **What it is:** Hackers using PowerShell commands to control systems
- **How we detect it:** Look for suspicious command patterns
- **Example:** Encoded PowerShell commands = ALERT!

4. Advanced Persistent Threats (APTs)

- **What it is:** Sophisticated, long-term attacks
- **How we detect it:** Connect multiple suspicious events
- **Example:** Failed login + suspicious process + data transfer = APT ALERT!

Risk Scoring System (How Dangerous is Each Event?)

1-2: LOW RISK

- Normal user activity
- Regular system operations
- Example: User logs in during business hours

- 3-4: MEDIUM RISK
 - Slightly unusual activity
 - Worth monitoring
 - Example: Login from new location
- 5-6: HIGH RISK
 - Suspicious patterns detected
 - Requires attention
 - Example: Multiple failed logins
- 7-8: CRITICAL RISK
 - Likely attack in progress
 - Immediate response needed
 - Example: Successful login after many failures
- 9-10: EMERGENCY
 - Confirmed attack
 - System may be compromised
 - Example: Multiple attack stages detected

Common Terms Explained

Technical Terms Made Simple

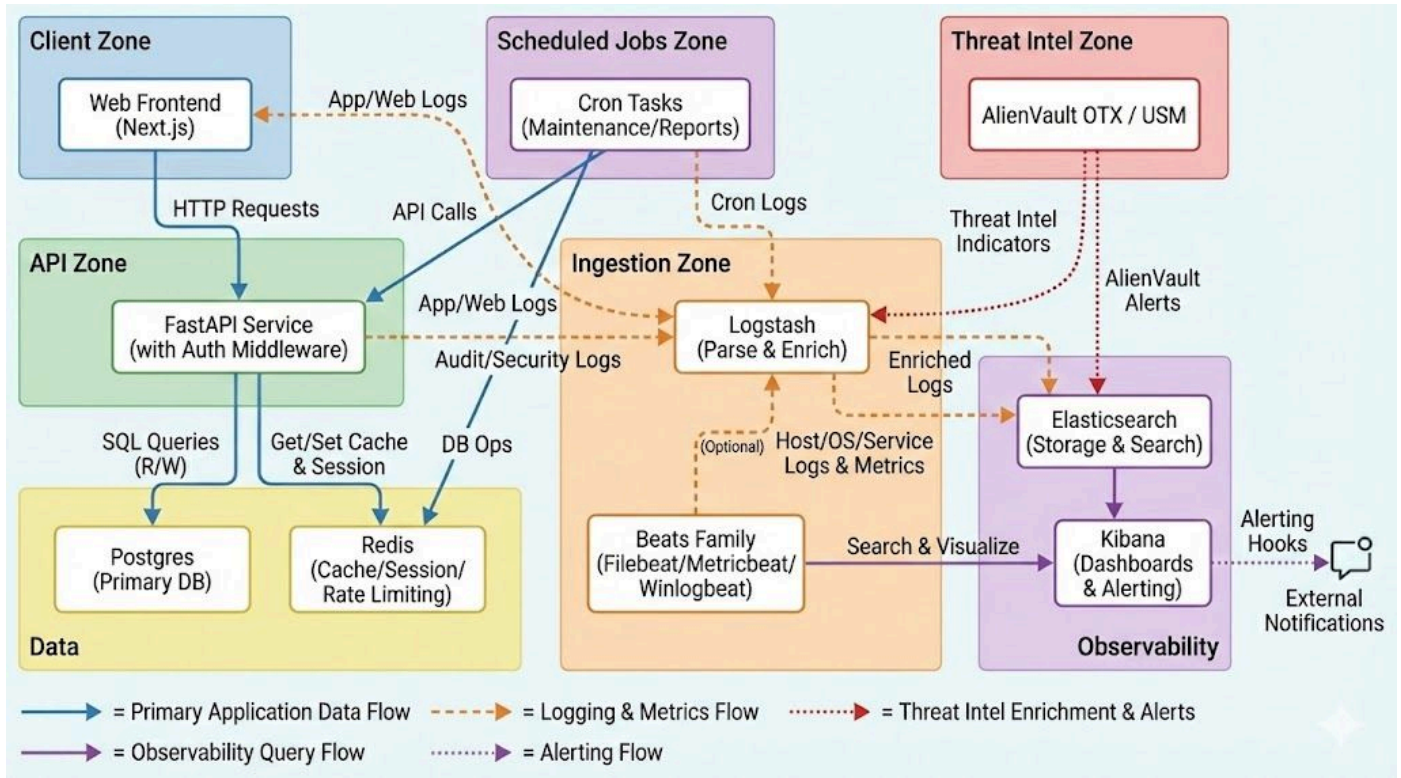
Technical Term	Simple Explanation	Real-World Example
API	A way for computers to talk to each other	Like a waiter who takes your order to the kitchen
Database	A place where information is stored permanently	Like a filing cabinet with organized folders
Cache	Temporary storage for quick access	Like keeping frequently used tools on your desk
Index	A way to find information quickly	Like the index at the back of a book
Log	A record of what happened	Like a diary that computers write in
Query	A question asked to a database	Like asking a librarian to find specific books
Token	A digital key that proves who you are	Like a temporary badge that gets you into a building

Technical Term	Simple Explanation	Real-World Example
Encryption	Scrambling data so only authorized people can read it	Like writing in secret code

Security Terms Made Simple

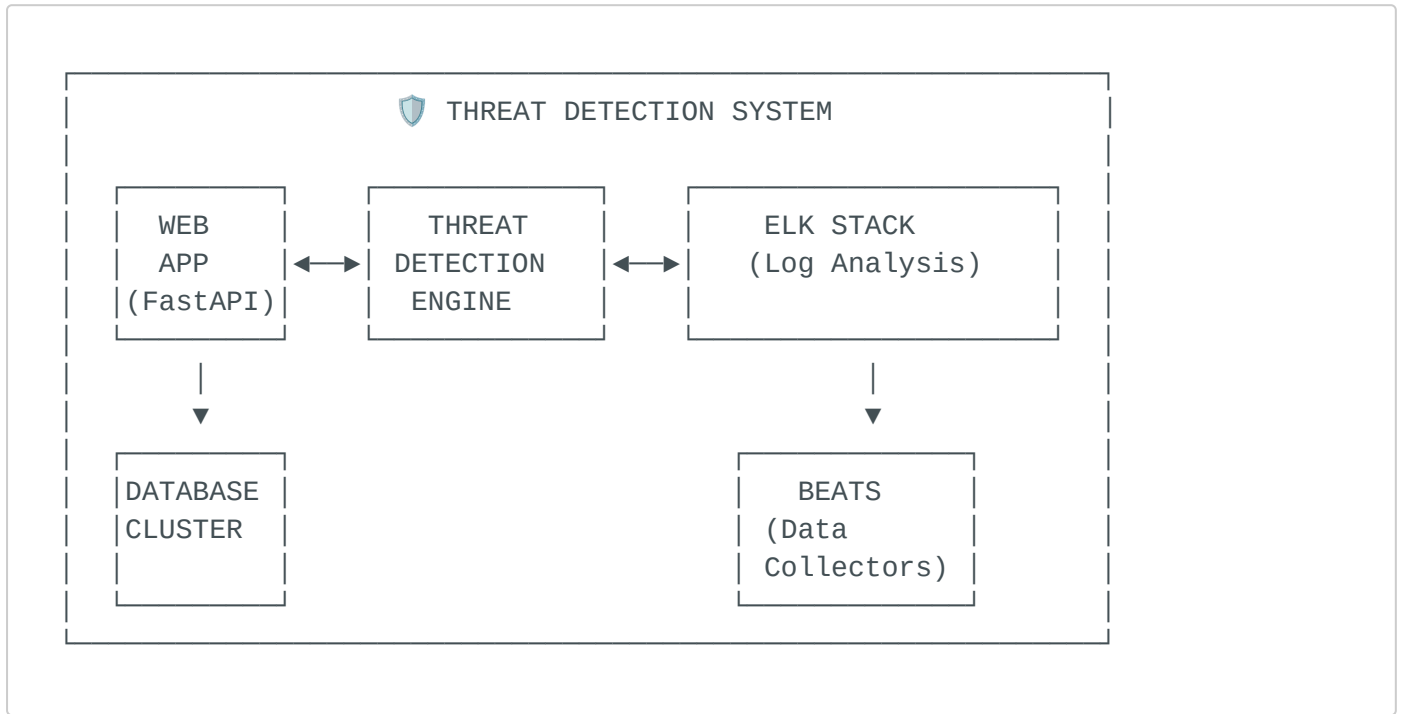
Security Term	Simple Explanation	Real-World Example
Threat	Something that could harm your system	Like a burglar trying to break into your house
Vulnerability	A weakness that could be exploited	Like leaving your front door unlocked
Attack	An actual attempt to harm your system	Like someone actually breaking into your house
Brute Force	Trying many passwords until one works	Like trying every key on a keyring
Malware	Harmful software	Like a computer virus
Phishing	Tricking people into giving away information	Like a fake email pretending to be your bank

2. System Architecture



High-Level Architecture

Think of our Advanced Threat Detection System like a **smart security guard** for computers. Just like how a security guard watches multiple cameras, our system watches computer logs from many different sources to catch bad guys (hackers) trying to break in.



What Does Each Part Do?

1. The Web App (FastAPI)

- **What it is:** The main application that users interact with
- **What it does:**
 - Manages user accounts and login
 - Handles todo lists (the main app feature)
 - Shows security alerts and threat information
 - Provides API endpoints for other systems
- **Like in real life:** The reception desk at a building where people check in

2. Threat Detection Engine

- **What it is:** Smart software that analyzes security data
- **What it does:**
 - Watches for suspicious login attempts
 - Detects when someone tries to steal data
 - Finds malicious PowerShell commands
 - Connects the dots between different attacks
- **Like in real life:** A detective who looks at clues and solves crimes

3. ELK Stack (The Brain)

- **What it is:** Three connected systems that store and analyze logs
- **What it does:** Collects, processes, and visualizes security data
- **Like in real life:** A security command center with monitors showing everything

4. Database Cluster

- **What it is:** Where all the data is stored
- **What it does:** Keeps user accounts, todos, and cached data safe
- **Like in real life:** A filing cabinet system with different drawers

5. Beats (Data Collectors)

- **What it is:** Tiny programs that collect data from computers
- **What it does:** Gathers logs from different sources and sends them to ELK
- **Like in real life:** Security cameras placed around a building

How Data Flows Through the System

Step 1: Data Collection

Computer Events → Beats → Logstash → Elasticsearch

Step 2: Analysis

Elasticsearch → Threat Detection Engine → Alerts

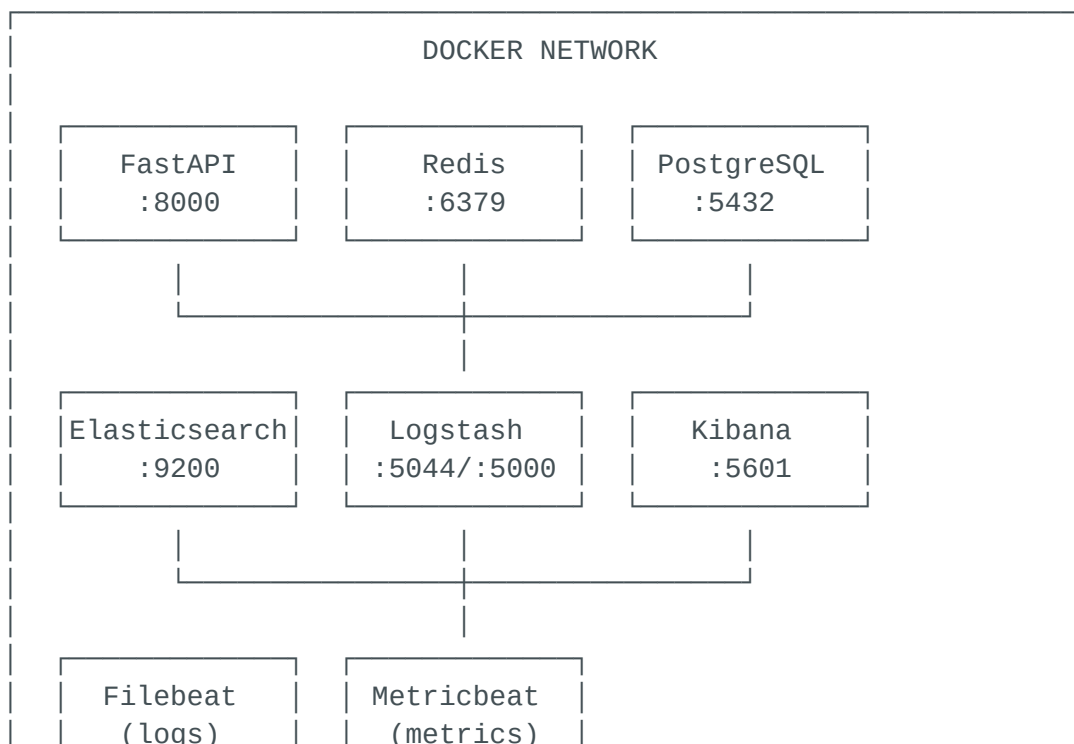
Step 3: User Interaction

User → FastAPI → Database/ELK → Response

Detailed Data Flow

1. **Something happens on a computer** (login, file access, network traffic)
2. **Beats collect the event** and send it to Logstash
3. **Logstash processes the data** (cleans it up, adds extra info)
4. **Elasticsearch stores the data** in searchable format
5. **Threat Detection Engine analyzes** the data for suspicious patterns
6. **If threats are found**, alerts are sent to administrators
7. **Kibana shows visual dashboards** of what's happening
8. **Users interact through FastAPI** to see alerts and manage settings

Physical Architecture (How Components Connect)



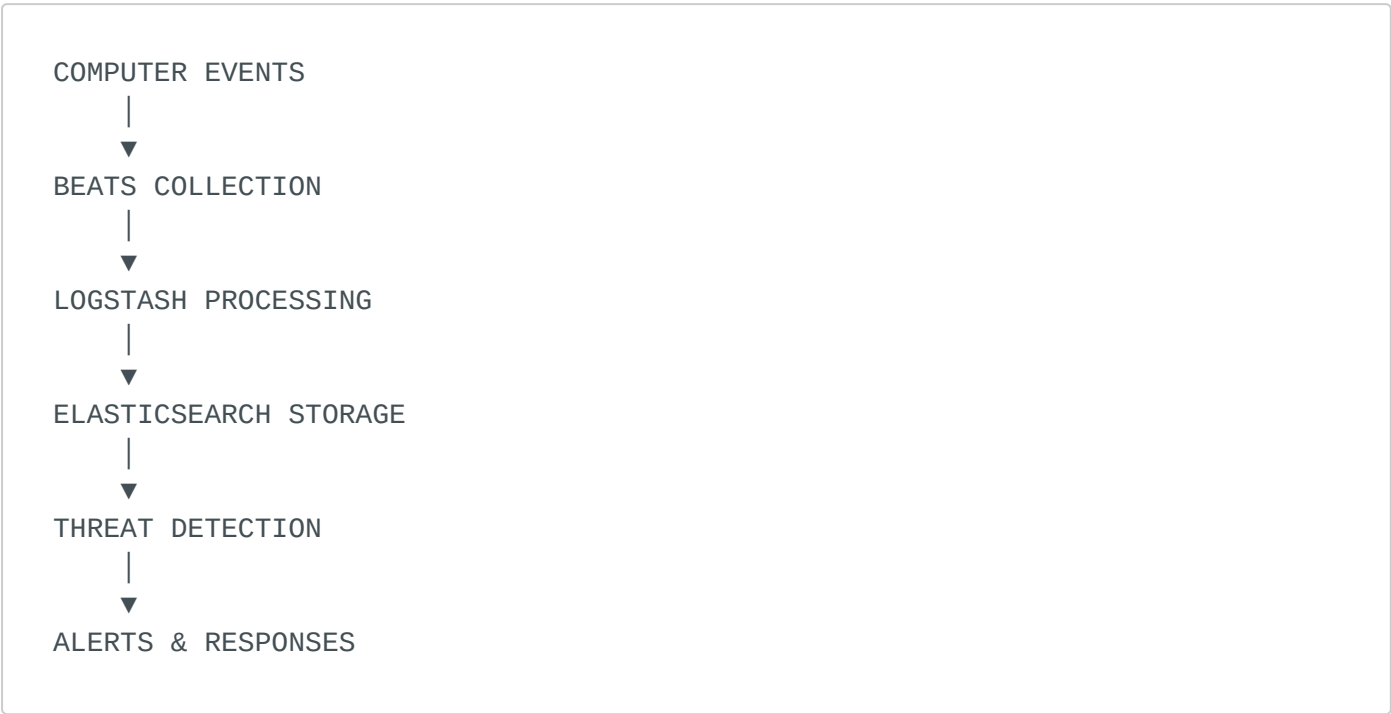
Component Interactions

Complete Data Flow Diagrams

This section shows exactly how data moves between all components in our Advanced Threat Detection System. Think of it as a **detailed map** showing how information travels from one place to another.

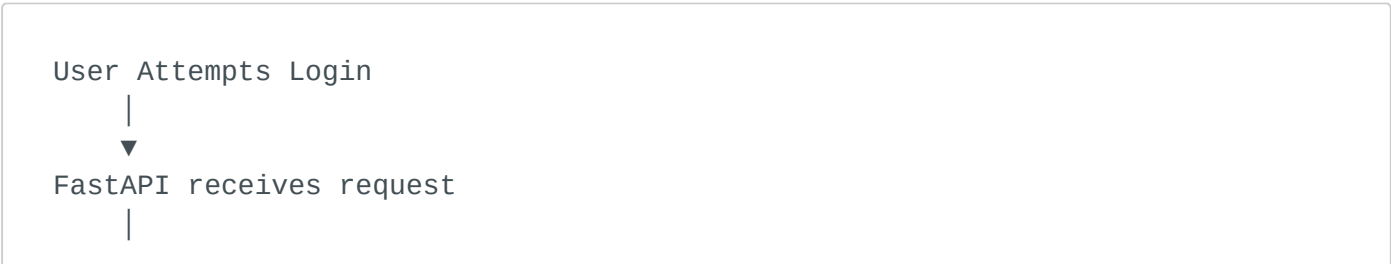
Data Collection Flow

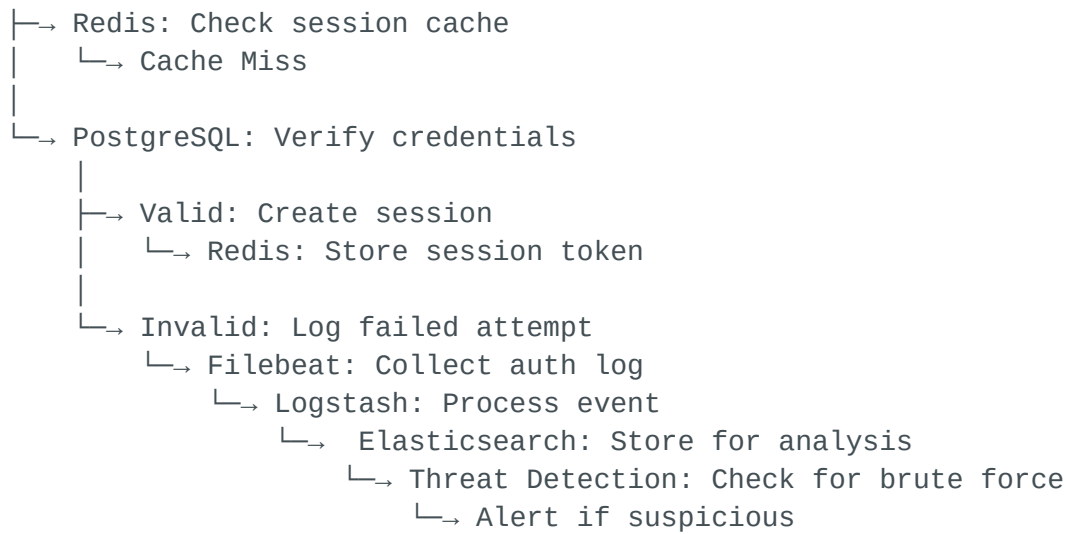
From Computer Events to Security Alerts



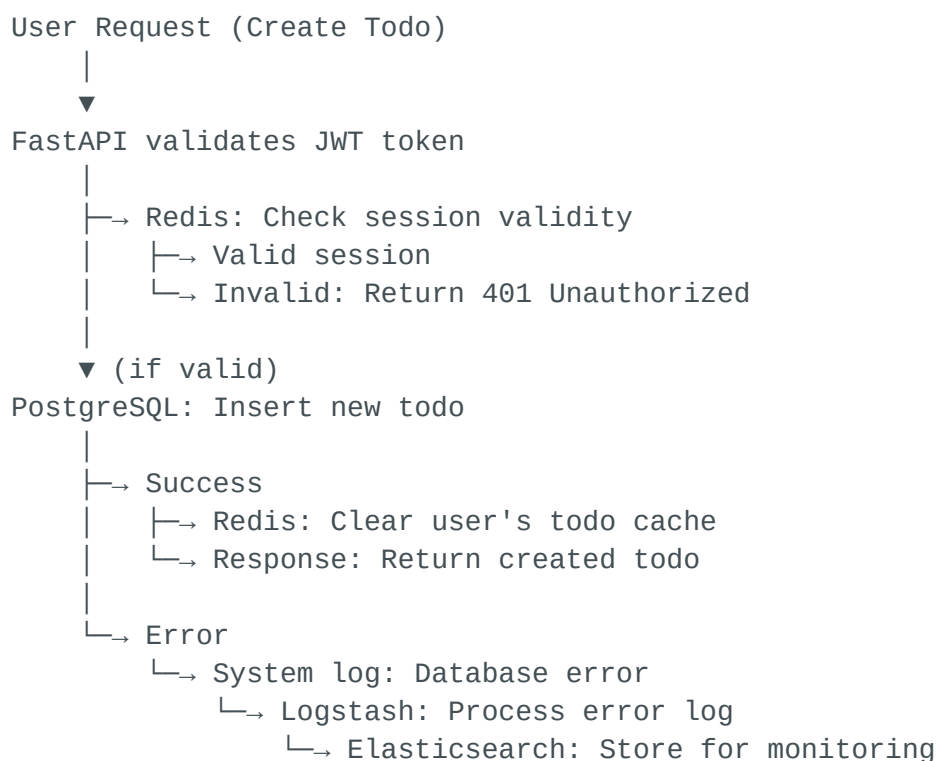
Detailed Component Interactions

1. User Authentication Flow

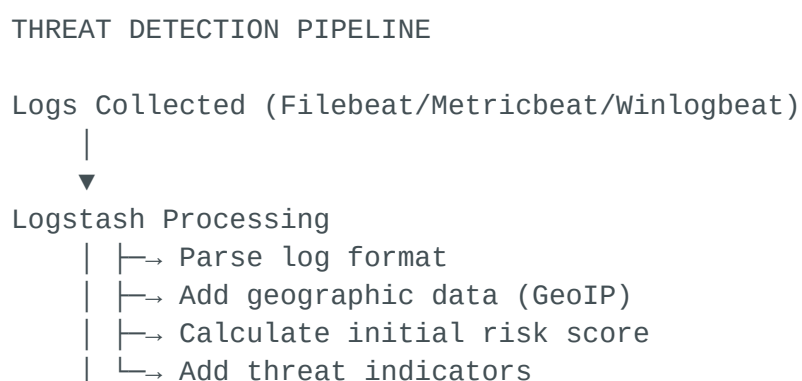




2. Todo CRUD Operations Flow



3. Security Threat Detection Flow



```
OUTPUT:
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "security-logs-%{+YYYY.MM.dd}"
  }
}
```

ELK Stack Guide

What is the ELK Stack?

ELK stands for **Elasticsearch**, **Logstash**, and **Kibana**. Think of it like a team of three friends who work together to help you understand what's happening in your computer systems:

- **Elasticsearch** = The librarian who organizes and stores millions of books (logs)
- **Logstash** = The translator who makes sure all books are written in the same language
- **Kibana** = The artist who draws pictures to help you understand the books

Elasticsearch - The Data Storage Expert

What is Elasticsearch?

Elasticsearch is like a **super-smart filing cabinet** that can store millions of documents (logs) and find exactly what you're looking for in seconds.

How Does it Work?

Think of it like Google for your logs:

1. Store millions of security events
2. Search through them instantly
3. Find patterns and connections
4. Answer questions like:
 - "Show me all failed logins from China"
 - "Find suspicious PowerShell commands"
 - "What happened between 2 AM and 4 AM?"

Key Features

Feature	What it Does	Example
Indexing	Organizes data for fast searching	Like a library catalog system
Full-Text Search	Find any word in millions of logs	Search for "failed password"
Aggregations	Count and summarize data	"How many attacks per country?"
Real-time	Shows data as it happens	Live threat detection

Configuration in Our System

```
# From docker-compose.yml
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:8.11.0
  ports:
    - "9200:9200" # Main API port
    - "9300:9300" # Internal cluster communication
  environment:
    - discovery.type=single-node
    - xpack.security.enabled=false
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
```

Data Structure (Indices)

Our system creates these "filing cabinets":

security-auth-logs-*	→ Login/logout events
security-network-logs-*	→ Network traffic and firewall
security-audit-logs-*	→ File access and system changes
security-alerts-*	→ Detected threats and alerts
windows-security-logs-*	→ Windows-specific events
application-logs-*	→ Our FastAPI application logs

Example Data in Elasticsearch

```
{
  "@timestamp": "2024-01-15T10:30:00Z",
```

```
"security_event": "authentication_failure",
"src_ip": "203.0.113.42",
"user_name": "admin",
"geo": {
  "country": "China",
  "city": "Beijing"
},
"risk_score": 8,
"threat_indicators": ["external_ip", "privileged_account"]
}
```

Logstash - The Data Processor

What is Logstash?

Logstash is like a **smart factory worker** who takes messy, different-formatted logs from various sources and cleans them up into a standard format that Elasticsearch can understand.

How Does it Work?

INPUT → FILTER → OUTPUT

1. INPUT: Receives raw logs from Filebeat, Metricbeat, etc.
2. FILTER: Cleans, parses, and enriches the data
3. OUTPUT: Sends processed data to Elasticsearch

The Pipeline Process

Raw Log: "Jan 15 10:30:00 server1 sshd: Failed password for admin from 203.0.113.42"

After Logstash Processing:

```
{
  "timestamp": "2024-01-15T10:30:00Z",
  "host": "server1",
  "service": "sshd",
  "event_type": "authentication_failure",
  "username": "admin",
  "source_ip": "203.0.113.42",
  "geo_location": "China",
  "risk_score": 7
}
```

Configuration Ports

Port	Purpose	Who Connects
5044	Beats input	Filebeat, Metricbeat, Winlogbeat
5000	TCP input	Application logs, custom systems
9600	API/Monitoring	Health checks, statistics
514	Syslog UDP/TCP	Network devices, routers, firewalls
12201	GELF	Docker container logs

Processing Pipeline Example

```
# Logstash configuration snippet
input {
  beats {
    port => 5044
  }
  tcp {
    port => 5000
    codec => json
  }
}

filter {
  # Add geographic information
  geoip {
    source => "src_ip"
    target => "geo"
  }

  # Calculate risk score
  if [failed_attempts] > 5 {
    mutate {
      add_field => { "risk_score" => 8 }
    }
  }
}

output {
  elasticsearch {
    hosts => ["elasticsearch:9200"]
    index => "security-logs-%{+YYYY.MM.dd}"
  }
}
```

Kibana - The Visualization Dashboard

What is Kibana?

Kibana is like an **artist and detective combined** - it takes all the data from Elasticsearch and creates beautiful, easy-to-understand charts, graphs, and dashboards.

How Does it Work?

1. Connects to Elasticsearch
 2. Reads the stored data
 3. Creates visualizations:
 - Pie charts showing attack sources by country
 - Line graphs showing attacks over time
 - Heat maps of suspicious activity
 - Tables of top threats

Key Features

Feature	What it Does	Example Use
Discover	Search and explore raw data	Find specific security events
Visualize	Create charts and graphs	Show attacks by country
Dashboard	Combine multiple visualizations	Security operations center view
Canvas	Create infographic-style reports	Executive summary presentations
Maps	Show geographic data	World map of attack sources

Dashboard Examples

Security Operations Dashboard





Geographic Threat Map

World Map showing:
Red dots = High-risk attack sources
Yellow dots = Medium-risk sources
Green dots = Low-risk sources

Hover over any dot to see:

- Number of attacks
- Attack types
- Risk level
- Blocked/Allowed ratio

Configuration

```
# From docker-compose.yml
kibana:
  image: docker.elastic.co/kibana/kibana:8.11.0
  ports:
    - "5601:5601" # Web interface
  environment:
    - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
  depends_on:
    - elasticsearch
```

Creating Security Index Patterns

When you first set up Kibana, create these index patterns:

1. **security-*** - All security-related logs
2. **security-auth-logs-*** - Authentication events only
3. **windows-security-logs-*** - Windows-specific events
4. **security-alerts-*** - Detected threats and alerts

How ELK Components Work Together

Data Flow Example: Detecting a Brute Force Attack

```
Step 1: EVENT OCCURS
Someone tries to log in with wrong password 5 times

Step 2: FILEBEAT COLLECTS
Filebeat sees the authentication failure in /var/log/auth.log

Step 3: LOGSTASH PROCESSES
- Parses the log format
- Extracts IP address, username, timestamp
- Adds geographic location (GeoIP lookup)
- Calculates risk score (5 failures = high risk)
- Adds threat indicators

Step 4: ELASTICSEARCH STORES
Stores the processed event in security-auth-logs-2024.01.15 index

Step 5: KIBANA VISUALIZES
- Updates "Failed Logins" counter on dashboard
- Adds red dot to world map
- Triggers alert visualization
- Updates attack timeline graph

Step 6: THREAT DETECTION ENGINE
- Queries Elasticsearch for similar events
- Detects brute force pattern
- Sends alerts to Slack and email
```

4. Setup & Configuration

Complete Setup Guide

This guide will take you through setting up the Advanced Threat Detection System from scratch.

Prerequisites

System Requirements

Requirement	Minimum	Recommended for Production
OS	Linux, macOS, Windows 10+	Linux (Ubuntu 20.04+)
RAM	8 GB	32 GB+

Requirement	Minimum	Recommended for Production
CPU	4 cores	16 cores+
Storage	50 GB free	500 GB SSD

Required Software

- **Docker** (version 20.10+)
- **Docker Compose** (version 2.0+)
- **Git** (for cloning the repository)
- **curl & jq** (optional, for testing)

Installation Steps

Step 1: Clone the Repository

```
git clone https://github.com/rohansen856/elk-stack-monitoring
cd elk-stack-monitoring
```

Step 2: Configure Environment Variables

```
# Copy the example environment file
cp .env.example .env

# Edit the configuration file
nano .env
```

Key Configuration Areas:

- **Database:** `POSTGRES_USER`, `POSTGRES_PASSWORD`
- **Security:** `SECRET_KEY` (Generate a strong random string)
- **ELK Stack:** `ELASTICSEARCH_URL`
- **Alerting:** `SLACK_WEBHOOK_URL`, `SMTP_SERVER`

Step 3: Start the System

```
# Start all services in detached mode
docker-compose up -d
```

```
# Check status (wait for "healthy" status)
docker-compose ps
```

Step 4: Initialize the Database

```
# Run database migrations
docker-compose exec app alembic upgrade head
```

Step 5: Verify Installation

```
# Check Elasticsearch Health
curl -u elastic:elastic123 http://localhost:9200/_cluster/health

# Check Application Health
curl http://localhost:8000/health
```

Security Credentials & Access

Default Credentials (Development Only)

IMPORTANT: Change these passwords immediately for production use.

Service	Username	Password	Role
Elasticsearch	elastic	elastic123	Superuser
Kibana System	kibana_system	I6aZ-X3eqk3Ty-5LGNWB	Internal System User
PostgreSQL	user	change_this_password	Database Owner
Test User	testuser	testpass123	Application User

Access Points

Service	URL	Description
Frontend	<code>http://localhost:3000</code>	Main User Interface
Backend API	<code>http://localhost:8000</code>	REST API & Swagger Docs
Kibana	<code>http://localhost:5601</code>	Analytics Dashboard
Elasticsearch	<code>http://localhost:9200</code>	Database (Requires Auth)

Security Features Active

1. **HTTP Authentication:** All Elasticsearch API calls require username/password.
2. **Role-Based Access:** Using built-in users (`elastic`, `kibana_system`).
3. **Secure Internal Communication:** All ELK components authenticate with each other.

Production Security Checklist

1. **Change Default Passwords:**

```
docker compose exec elasticsearch bin/elasticsearch-reset-password -u elastic -i
```

2. **Enable HTTPS:** Configure Nginx with valid SSL certificates.
3. **Restrict Network Access:** Ensure ports 9200, 5432, and 6379 are not exposed to the public internet.

Troubleshooting Common Issues

Issue 1: Services Won't Start

Error: "Cannot connect to the Docker daemon" **Solution:** Ensure Docker is running (`sudo systemctl start docker`) and your user is in the docker group.

Issue 2: Elasticsearch Won't Start

Error: Container exits immediately or restarts loop. **Common Cause:**

`vm.max_map_count` is too low. **Solution:**

```
sudo sysctl -w vm.max_map_count=262144  
echo 'vm.max_map_count=262144' | sudo tee -a /etc/sysctl.conf
```

Issue 3: Kibana Shows No Data

Problem: Dashboards are empty. **Solution:**

1. Check if Logstash is processing data: `curl http://localhost:9600/_node/stats/pipelines`
2. Create Index Patterns in Kibana: `security-*`, `security-auth-logs-*`.
3. Generate test data by using the application (login/logout).

Issue 4: Database Connection Failed

Error: FastAPI logs show connection refused. **Solution:** Wait for PostgreSQL to fully initialize (can take 30-60s on first run). Check `docker-compose logs db`.

Emergency Reset

If everything fails and you need to start fresh (WARNING: Deletes all data):

```
docker-compose down -v  
docker system prune -a  
docker-compose up -d
```

5. Deployment Guides

Kubernetes Deployment

This guide covers deploying the system on Kubernetes using Kind or Minikube.

Quick Start (Automated)

```
# Clone and setup
git clone https://github.com/rohansen856/elk-stack-monitoring
cd elk-stack-monitoring
chmod +x scripts/*.sh

# Deploy automatically
./scripts/k8s-deploy.sh
```

The script will:

1. Detect/Start Cluster (Kind/Minikube)
2. Build & Load Images
3. Install Ingress Controller
4. Deploy All Components
5. Run Migrations

Manual Deployment Steps

1. Cluster Setup

```
# Kind
kind create cluster --config scripts/kind-config.yaml

# Minikube
minikube start --cpus=4 --memory=8192 --disk-size=50g
```

2. Build & Load Images

```
docker build -t ps25238-app:latest .
docker build -t ps25238-frontend:latest ./website

# Load into Kind
kind load docker-image ps25238-app:latest
kind load docker-image ps25238-frontend:latest
```

3. Deploy Resources


```
# Apply Kustomize configuration
kubectl apply -k k8s/base/

# Wait for rollout
kubectl wait --for=condition=available --timeout=300s deployment/backend -n
threat-detection
```

Accessing Applications

Kind (Port Forwarding):

```
kubectl port-forward -n threat-detection svc/frontend 3000:3000
kubectl port-forward -n threat-detection svc/backend 8000:8000
kubectl port-forward -n threat-detection svc/kibana 5601:5601
```

Minikube:

```
minikube service frontend-loadbalancer -n threat-detection --url
```

Architecture

- **Namespace:** `threat-detection`
- **Data:** PostgreSQL (5GB), Redis (1GB), Elasticsearch (10GB)
- **Apps:** FastAPI Backend (HPA enabled), Next.js Frontend
- **Monitoring:** Filebeat & Metricbeat DaemonSets

AWS EC2 Deployment

Quick Start (One-Command)

Prerequisites: Ubuntu 20.04+ EC2 instance with Port 80 open.

```
# SSH into instance
ssh -i key.pem ubuntu@your-ec2-ip
```

```
# Run setup script
git clone https://github.com/rohansen856/elk-stack-monitoring.git
cd elk-stack-monitoring
chmod +x scripts/aws-ec2-setup.sh
./scripts/aws-ec2-setup.sh
```

Manual Setup Steps

1. Install Docker & Compose:

```
curl -fsSL https://get.docker.com | sudo bash
sudo usermod -aG docker $USER
```

2. Configure Environment:

```
cp .env.example .env
# Get Public IP
EC2_IP=$(curl -s http://169.254.169.254/latest/meta-data/public-ipv4)
# Update Frontend Config
sed -i "s|http://localhost/frontend|http://$EC2_IP/frontend|g"
website/.env
```

3. Build & Run:

```
docker compose build --no-cache
docker compose up -d
```

Access Points (AWS)

- **Frontend:** http://<EC2_IP>/frontend
- **Backend API:** http://<EC2_IP>/backend/docs
- **Kibana:** http://<EC2_IP>/monitoring

Maintenance

```
# View Logs
docker compose logs -f
```

```
# Restart Services
docker compose restart

# Update Application
git pull
docker compose build
docker compose up -d
```

6. Operations & API

Monitoring & APT Detection

This section details how the system detects Advanced Persistent Threats (APTs) through centralized log analysis.

Log Sources & Detection Capabilities

Log Source	What it Detects	Key Events
Authentication	Brute force, unusual locations	Failed logins, sudo usage
Network Traffic	Data exfiltration, C2 comms	Blocked connections, large transfers
Process Creation	Malware execution, persistence	PowerShell scripts, new services
File Access	Data theft, tampering	Access to sensitive files

Risk Scoring System

Events are automatically assigned a risk score (1-10):

- **1-2 (Low):** Normal activity
- **3-4 (Medium):** Blocked connections, single failures
- **5-6 (High):** Multiple failures, privilege escalation
- **7-8 (Critical):** Suspicious processes, external admin access

- **9-10 (Emergency):** Confirmed malicious activity

APT Detection Scenarios

1. Credential Stuffing

Pattern: High volume of failed logins from one IP against multiple users. **Indicators:** `authentication_failure` + High Risk Score + External IP.

2. Lateral Movement

Pattern: Admin login to multiple internal hosts off-hours. **Indicators:** `authentication_success` + Network Logon + Off-hours.

3. Data Exfiltration

Pattern: Large outbound data transfer via HTTPS. **Indicators:** `network_security` + >100MB Data + Off-hours.

Alerting Rules (ES|QL)

These rules are pre-configured for Kibana's Stack Management.

1. Brute Force Detection

Query:

```
FROM security-auth-logs-*
| WHERE event.action == "authentication_failure"
| STATS failed_attempts = COUNT(*) BY source.ip, user.email
| WHERE failed_attempts >= 3
| EVAL threat_level = "HIGH", attack_type = "Brute Force"
```

2. PowerShell Attack Detection

Query:

```
FROM security-powershell-logs-*
| WHERE event.action == "powershell_execute"
| WHERE process.command_line RLIKE ".*(-enc|-EncodedCommand|base64).*"
| EVAL threat_level = "CRITICAL", attack_type = "PowerShell LOLBINS"
```

3. Data Exfiltration

Query:

```
FROM security-network-logs-*
| WHERE network.bytes_out > 1048576
| STATS total_bytes = SUM(network.bytes_out) BY source.ip, destination.ip
| WHERE (total_bytes / 1048576) > 5
| EVAL threat_level = "MEDIUM", attack_type = "Large Data Transfer"
```



API Documentation

The system exposes a REST API via FastAPI.

Base URL: <http://localhost:8000> **Docs:** <http://localhost:8000/docs>

Authentication

Include the JWT token in the header: **Authorization: Bearer <your-jwt-token>**

Key Endpoints

User Management

- **POST** [/api/v1/users/register](#): Create account
- **POST** [/api/v1/users/login](#): Get JWT token
- **GET** [/api/v1/users/me](#): Get profile

Security Monitoring

- **GET /api/v1/security/threats/brute-force**: Detect brute force attacks
- **GET /api/v1/security/threats/data-exfiltration**: Monitor data leaks
- **GET /api/v1/security/threats/apt-correlation**: Analyze kill-chain patterns

System Health

- **GET /health**: Check database & Redis status
- **GET /metrics**: Prometheus metrics

Example Usage

Login:

```
curl -X POST "http://localhost:8000/api/v1/users/login" \  
  -H "Content-Type: application/json" \  
  -d '{"email":"admin@example.com","password":"password"}'
```

Check Threats:

```
curl "http://localhost:8000/api/v1/security/threats/brute-force" \  
  -H "Authorization: Bearer <token>"
```